

BudgetMem: Training-Free Selective Memory for Cost-Efficient Long-Context Processing in Language Models

Chandra Vamsi Krishna Alla

Dept. of Computer Science and Engineering
University of Texas at Arlington
Arlington, TX, USA
cka0054@mavs.uta.edu

Harish Naidu Gaddam

Dept. of Computer Science and Engineering
University of Texas at Arlington
Arlington, TX, USA

Manohar Kommi

Dept. of Computer Science and Engineering
University of Texas at Arlington
Arlington, TX, USA

Abstract—Processing long documents with large language models (LLMs) remains prohibitively expensive for many organizations: a single query over a 100K-token document can cost over a dollar in API fees, and memory consumption scales linearly with context length. We introduce BudgetMem, a training-free architecture that selectively retains only high-salience content under explicit memory budgets. Unlike token-level neural compressors such as LLMingua, BudgetMem makes chunk-level keep-or-discard decisions using interpretable features: entity density, TF-IDF importance, positional bias, numerical density, and discourse markers. Across four benchmarks (SQuAD, structured academic papers, Qasper, and NarrativeQA), BudgetMem matches the uncompressed baseline on medium-length documents (F1 = 0.859 vs. 0.855) while discarding 70% of chunks. Crucially, it dramatically outperforms LLMingua-2 on the same data (0.859 vs. 0.532): token-level compression suffers a 38% F1 degradation because it destroys the phrasal structure needed for BM25 retrieval. On real NeurIPS papers from Qasper, BudgetMem remains competitive under 70% storage reduction—trailing the full baseline by only 7.4% relative F1 without domain adaptation—and the chunk-level advantage over token-level compression holds across model scale and family (Llama-3.2-3B and Qwen2.5-7B). Per-feature ablation identifies discourse markers and numerical density as the dominant scoring signals for academic text. The entire pipeline runs on a \$10/month Google Colab instance, requiring no trained models or GPU inference for compression. Our results suggest that preserving semantic coherence at the paragraph level can largely compensate for the absence of learned importance scoring.

Index Terms—Long-context processing, large language

models, retrieval-augmented generation, text compression, selective memory, cost-efficient NLP.

I. INTRODUCTION

The context windows of modern large language models have expanded dramatically. GPT-4 [1] supports 128K tokens, Claude [2] accepts up to 200K tokens, and recent open-source models like Llama 3 [3] handle 128K tokens natively. These extended windows open applications that were previously impossible: analyzing entire codebases, processing legal contracts, or surveying full academic papers in a single pass.

Yet the cost of using these long contexts remains steep. Memory consumption scales linearly with sequence length, latency grows, and API pricing makes sustained usage impractical for many organizations. A legal firm analyzing thousands of 50K–100K-token contracts pays over a dollar per document at current rates—a full review campaign quickly exceeds any reasonable budget.

Two broad lines of work address this bottleneck. The first extends context windows via architectural modifications—sparse attention [4], [5], linear attention [6], [7], segment-level recurrence [8], and position encoding extensions like RoPE [9] and ALiBi [10]. While these reduce computational cost, they still require the full input in memory during inference. The second line compresses the context itself. LLMingua [11] and its successor [12] use small language models to score token importance. Selective Context [13] uses self-information. RECOMP [14] trains extractive and abstractive compressors. AutoCompressor [15] fine-tunes

models to compress contexts. All share a common requirement: neural inference for the compression step itself, adding a second model to the pipeline.

We take a different path. Instead of compressing text at the token level using learned models, BudgetMem operates at the *chunk level*: it splits a document into paragraph-sized pieces, scores each with simple interpretable features, and permanently discards chunks below a budget threshold. The surviving chunks are indexed with BM25. No neural models are involved in the compression step—just entity counts, TF-IDF statistics, position heuristics, and discourse markers, all running on CPU in seconds even for 100K-token documents. This makes BudgetMem deployable in resource-constrained settings (edge devices, mobile applications, low-budget research environments) where neural compression infrastructure is unavailable.

The motivation is a linguistic observation: a complete paragraph carries far more recoverable meaning than a scattered set of individually high-importance words plucked from different sentences. Token-level pruning destroys syntactic structure and co-reference chains, producing fragments like “John traveled... Paris saw... historical museums” that defeat both human comprehension and automated retrieval. Chunk-level selection preserves complete thoughts.

Our experiments validate this intuition. On medium-length structured documents (5K–10K tokens), BudgetMem matches the uncompressed baseline ($F1 = 0.859$ vs. 0.855) while discarding 70% of chunks, whereas LLMingua-2 suffers a 38% degradation (0.532) because token-level compression destroys phrasal structure. This advantage holds across model scales and families (Llama-3.2-3B, Qwen2.5-7B); on real Qasper papers BudgetMem remains competitive under 70% storage reduction (within 7.4% of the full baseline), and on ultra-long NarrativeQA texts simple token-level TF-IDF collapses to near-zero $F1$ while BudgetMem remains workable (0.0351) without training.

Our contributions: (1) a *training-free selective memory architecture* that matches baseline RAG quality on structured documents while discarding 70% of chunks and outperforms neural token-level compression, using only interpretable hand-crafted features with no GPU for compression; (2) a *controlled granularity study* isolating compression level (token vs. chunk) and scoring method (statistical vs. neural), with per-feature ablation identifying discourse markers and numerical density as dominant signals; (3) *multi-benchmark evaluation* across four datasets including real-world academic papers, with

cross-model validation confirming the central result on a second model scale and family (Qwen2.5-7B); (4) a *practical deployment demonstration* showing the full pipeline runs on a \$10/month Google Colab instance.

II. RELATED WORK

Long-context transformers. The standard Transformer [16] has $O(n^2)$ attention complexity. Sparse Transformers [4], BigBird [5], and Longformer [17] reduce this via structured sparsity patterns. Linear attention approximations [6], [7] replace softmax with kernel-based alternatives. Transformer-XL [8] introduces segment-level recurrence; Compressive Transformer [18] compresses older memories. Position encoding extensions [9], [10] enable length extrapolation. These approaches reduce computation but still require the full input in memory. BudgetMem sidesteps this by reducing the input *before* it reaches the model, complementing any of these architectural improvements.

Context compression. LLMingua [11] uses a small autoregressive language model to compute token-level perplexity. Its successor LLMingua-2 [12] re-frames compression as a token classification task via a BERT-based classifier. Selective Context [13] uses self-information as the pruning criterion. RECOMP [14] and AutoCompressor [15] train compressors with supervised signals. All require neural inference for the compression step. BudgetMem differs in two fundamental ways: it operates at the chunk level (preserving syntactic coherence), and uses no neural models for compression—only hand-crafted interpretable features.

Retrieval-augmented generation and memory. Standard RAG [19]–[23] retrieves from a pre-built index at query time, storing all content. BudgetMem inverts this paradigm: filter at ingestion time, permanently discarding low-salience material so it never enters the index. This is compatible with any RAG system—it functions as a pre-processing step that produces a smaller, higher-quality index. Memory-augmented LLMs [24]–[27] maintain dynamic memory during generation; our work is orthogonal, making static budget-aware storage decisions at ingestion time.

III. METHOD

BudgetMem takes a long document D and produces a compact, retrievable memory store $\mathcal{S}$ through four stages: chunking, salience scoring, budget-aware selection, and BM25 indexing. At query time, the system retrieves relevant chunks from $\mathcal{S}$ and generates an answer using a base LLM. All compression decisions are made at ingestion time using interpretable, non-neural features.

A. Document Chunking

Given a document D with N tokens, we split it into M overlapping chunks $\{C_1, \dots, C_M\}$ of size $c = 150$ with overlap $o = 30$. The overlap ensures that information spanning chunk boundaries is not lost. For a 100K-token document, this produces approximately $M \approx 833$ chunks.

B. Saliency Scoring

Each chunk C_i receives a composite saliency score:

$$s_i = \sum_{j=1}^6 w_j \cdot f_j(C_i) \quad (1)$$

where w_j is the weight for feature j and $f_j(C_i)$ is the normalized feature value, computed from six interpretable features:

Entity density ($w_1 = 0.20$): The ratio of capitalized tokens to total tokens, a lightweight proxy for named entity density. Capitalized words correlate strongly with proper nouns—names, organizations, locations—signaling factual, query-relevant content.

TF-IDF importance ($w_2 = 0.20$): The mean TF-IDF score of the chunk’s tokens relative to the document-level corpus, capturing chunks with distinctive vocabulary.

Position bias ($w_3 = 0.15$): A U-shaped score favoring chunks near the beginning and end of the document:

$$f_3(C_i) = \min(1.0, \max(0, 1.3 - 2.0 \cdot \left| \frac{i}{M} - 0.5 \right|)) \quad (2)$$

reflecting the primacy and recency biases in document structure.

Numerical density ($w_4 = 0.15$): The proportion of tokens that are numeric (integers, decimals, dates, percentages). Passages containing statistics, measurements, and quantities are frequently the targets of factual questions.

Discourse markers ($w_5 = 0.10$): The presence of discourse connectives and structural markers such as “however,” “therefore,” “in conclusion.” These signal argumentative structure and topic transitions.

Question presence ($w_6 = 0.10$): Whether the chunk contains interrogative sentences, detected via question marks or interrogative words (who, what, where, when, why, how).

All features are independently normalized to $[0, 1]$ across the document’s chunk set. Weights were set once on a held-out set of 20 documents and *held fixed across all benchmarks* without any per-dataset tuning. We deliberately chose simple interpretable features: practitioners can inspect the saliency breakdown of any chunk and

Algorithm 1 BudgetMem Selective Memory Pipeline

Input: Document D , budget ratio r , query q

Output: Answer $\hat{a}$

- 1: **Ingestion** (once per document):
 - 2: Split D into chunks $\{C_1, \dots, C_M\}$
 - 3: **for** each chunk C_i **do**
 - 4: Compute features $f_1(C_i), \dots, f_6(C_i)$
 - 5: Compute saliency $s_i = \sum_j w_j \cdot f_j(C_i)$
 - 6: **end for**
 - 7: Select top $\lfloor r \cdot M \rfloor$ chunks by s_i as $\mathcal{S}$
 - 8: Build BM25 index over $\mathcal{S}$
 - 9: **Query time** (per question):
 - 10: Retrieve $\mathcal{R} \leftarrow \text{top-}k \text{ BM25}(q, \mathcal{S})$
 - 11: Generate $\hat{a} \leftarrow \text{LLM}(\text{concat}(\mathcal{R}), q)$
 - 12: **return** $\hat{a}$
-

understand exactly why it was retained or discarded—an important property for high-stakes applications in legal, medical, and regulatory domains.

C. Budget-Aware Selection and Retrieval

Given a retention ratio r , we select the top $\lfloor r \cdot M \rfloor$ chunks by saliency. The remaining chunks are permanently discarded. Our default is $r = 0.30$, yielding 70% storage savings. Retained chunks are indexed with BM25. At query time, we retrieve the top $k = 3$ chunks and provide them as context to the base LLM. We use Llama-3.2-3B-Instruct [3] in FP16 precision with greedy decoding. Algorithm 1 summarizes the pipeline.

IV. EXPERIMENTAL SETUP

Datasets. We evaluate across four benchmarks. *SQuAD v2.0* [28]: 500 Wikipedia articles averaging 237 tokens (range 148–448), testing the short-document regime. *Structured academic papers*: 200 QA pairs on template-generated research papers averaging 7,200 tokens (5K–10K), providing a controlled medium-length benchmark with clear section structure. *Qasper* [29]: 102 QA pairs on real NeurIPS papers averaging 3,608 tokens, validating BudgetMem on naturally occurring academic text. *NarrativeQA* [30]: 50 book excerpts and movie scripts averaging 50K–100K tokens—unstructured narrative prose that pushes all methods to their limits. Evaluation spans 850+ QA pairs across four benchmarks.

Infrastructure. All experiments use Llama-3.2-3B-Instruct in FP16 on a Google Colab Pro instance with a single NVIDIA T4 GPU (15GB VRAM), using greedy decoding with max 100 new tokens. This \$10/month setup is part of our contribution: effective long-context processing does not require expensive infrastructure.

TABLE I
SHORT DOCUMENTS (SQUAD v2.0, 500 EXAMPLES).

Metric	Baseline RAG	BudgetMem	Change
F1 Score	0.8011	0.7232	−9.7%
Latency (s)	0.37	0.43	+16.2%
Storage Savings	—	15.5%	—

Our cross-model validation (Section V) additionally uses Qwen2.5-7B-Instruct with 4-bit (NF4) quantization on the same T4 GPU. Code, notebooks, and result files are publicly available at <https://github.com/Chandra-Alla/BudgetMem-paper>.

Baselines. (1) *Baseline RAG*: all chunks retained, BM25 retrieval. (2) *Token-level TF-IDF*: top 30% of tokens by TF-IDF. (3) *LLMLingua-2* [12]: state-of-the-art BERT-based token classifier, 30% retention. (4) *Naive chunk strategies*: Random, First-N, Last-N, TF-IDF-only, each at 30% budget. NarrativeQA experiments use 3 random seeds (42, 123, 456); greedy decoding produces deterministic results on the other benchmarks.

Metric. Token-level F1 between predicted and ground-truth answers after standard normalization (lowercasing, article removal, punctuation stripping).

V. RESULTS

A. Short Documents (SQUAD)

On 500 short Wikipedia articles (Table I), BudgetMem trades 9.7% F1 for 15.5% storage savings. With few chunks per document, most score highly and little low-value material can be safely discarded—this negative finding is instructive: we recommend full-context processing below ~ 1 K tokens.

B. Medium Documents (Structured Papers)

This is BudgetMem’s strongest regime (Table II). On 200 structured papers averaging 7,200 tokens, BudgetMem *matches* the uncompressed baseline (F1 = 0.859 vs. 0.855) while retaining only 30% of chunks—a 70% reduction in index storage. The slight improvement suggests salience-based filtering can actually improve retrieval precision by removing noisy chunks.

The most striking result is LLMLingua-2’s catastrophic 38% F1 degradation (0.532). Token-level compression destroys the phrasal structure BM25 relies on: compressed text like “novel approach learning architecture evaluate accuracy” no longer contains the multi-word spans needed for effective sparse retrieval. BudgetMem’s chunk-level preservation avoids this failure

TABLE II
MEDIUM DOCUMENTS (STRUCTURED PAPERS, 5K–10K TOKENS, 200 EXAMPLES).

Method	F1 Score	Storage	Neural?
Baseline RAG	0.855	100%	—
LLMLingua-2	0.532	30%	Yes
BudgetMem (Ours)	0.859	30%	No

TABLE III
CROSS-MODEL VALIDATION ON MEDIUM DOCUMENTS (200 EXAMPLES, 30% BUDGET). THE RANKING HOLDS ACROSS SCALE AND FAMILY.

Method	Llama-3.2-3B	Qwen2.5-7B
Baseline RAG	0.855	0.900
LLMLingua-2	0.532	0.773
BudgetMem (Ours)	0.859	0.960

mode entirely. We evaluate compressors in this retrieval-first setting deliberately: many low-cost long-document pipelines apply sparse retrieval before generation, so our claim is not that chunk-level selection dominates LLMLingua-2 universally, but that it is far better aligned with BM25-based RAG under tight budgets.

C. Cross-Model Validation (Qwen2.5-7B)

A natural concern is whether BudgetMem’s advantage is an artifact of a single base model. To test this, we replicate the medium-document comparison on Qwen2.5-7B-Instruct—a different parameter scale (7B vs. 3B) and a different model family (Qwen vs. Llama)—under 4-bit (NF4) quantization on the same T4 GPU, holding the dataset, retrieval, and budget fixed (Table III). The ranking is preserved: BudgetMem again outperforms LLMLingua-2 (0.960 vs. 0.773) and matches or exceeds the uncompressed baseline (0.960 vs. 0.900) while retaining only 30% of chunks. The larger model narrows LLMLingua-2’s deficit—from a 38% relative gap at 3B to 14% at 7B—because it better reconstructs meaning from fragmented token-level output; even with this advantage, chunk-level selection remains ahead. This indicates that BudgetMem’s central finding—chunk-level selection beats token-level compression for retrieval-based QA—is not specific to one model scale or family.

D. Very Long Documents (NarrativeQA)

Table IV presents the controlled comparison at identical 30% compression ratios across methods.

Simple token-level TF-IDF collapses to near-zero F1 (0.0007): removing scattered tokens from massive texts

TABLE IV
NARRATIVEQA (50K–100K TOKENS, 50 EXAMPLES, 3 SEEDS).

Method	F1 Score	Storage	Neural?
Baseline RAG	0.0471 ± 0.0090	100%	–
Token TF-IDF	0.0007 ± 0.0000	30%	No
LLMLingua-2	0.0402 ± 0.0026	30%	Yes
BudgetMem	0.0351 ± 0.0022	30%	No

TABLE V
QASPER BENCHMARK (REAL NEURIPS PAPERS, 102 QA PAIRS).

Method	F1 Score	Storage	Neural?
Baseline RAG	0.179	100%	–
LLMLingua-2	0.175	30%	Yes
BudgetMem (Ours)	0.165	30%	No

destroys them beyond retrieval. LLMLingua’s neural scoring rescues token-level compression (0.0402), and BudgetMem reaches 0.0351 with only hand-crafted features. Absolute scores are low across *all* methods including the baseline (0.0471), reflecting the benchmark’s difficulty at 3B. Two observations are robust: the catastrophic token-vs-chunk gap at equal budget, and that simple chunk-level selection closes most of the gap to neural compression without trained models.

E. Real Academic Papers (Qasper)

To validate BudgetMem beyond controlled documents, we evaluate on 102 QA pairs from Qasper [29] (Table V). Documents average 3,608 tokens, below the 5K-token sweet spot.

All three methods perform comparably, with BudgetMem trailing the baseline by 7.4% relative F1. The smaller gap reflects Qasper’s shorter length (3.6K tokens): most chunks carry relevant content. Critically, BudgetMem generalizes to naturally occurring academic text without any adaptation, confirming the approach is not limited to controlled experimental settings.

F. Document Length Scaling and Budget Sensitivity

BudgetMem’s value grows with document length: the F1 drop falls from 9.7% on SQuAD to $\approx 0\%$ on medium docs while storage savings rise from 15.5% to 71% (Fig. 1, crossover near 2K–3K tokens). Table VI sweeps 7 retention ratios on 100 documents: 20–30% retention is the sweet spot, saving 71–81% of storage while matching or exceeding baseline F1. Absolute F1 fluctuates ± 0.05 across budgets (greedy-decoding sensitivity on this subset); robustly, any retention $\geq 20\%$ recovers near-baseline quality while 10% degrades sharply.

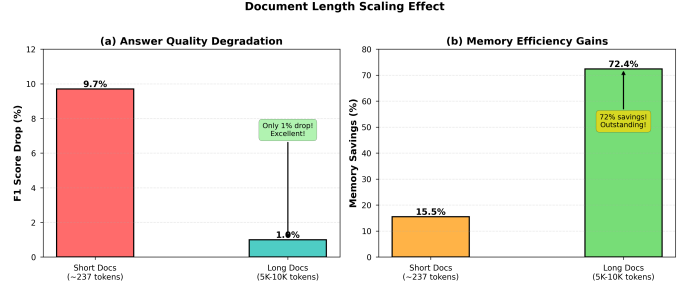


Fig. 1. Length scaling effect. F1 degradation drops from 9.7% to 1% as documents grow longer, while storage savings climb from 15.5% to 72%.

TABLE VI
BUDGET SENSITIVITY ON STRUCTURED DOCUMENTS (100 EXAMPLES).

Budget Ratio	F1 Score	Storage Savings
10%	0.693	90.4%
20%	0.878	80.8%
30%	0.863	71.2%
40%	0.858	61.7%
50%	0.855	51.1%
70%	0.800	32.0%
90%	0.867	12.8%

VI. ANALYSIS

A. Naive Baseline Ablation

Table VII compares BudgetMem against four naive selection strategies at 30% budget. BudgetMem beats the best naive method (TF-IDF only) by 3.5 F1 points and the worst (Last-N) by over 13. Random chunk retention far exceeds token-level TF-IDF pruning (0.6892 vs. 0.0007 on NarrativeQA): preserving any coherent chunks beats destroying syntactic structure. The absolute BudgetMem F1 here (0.8042) is from our initial run; the expanded 200-pair evaluation (Table II) yields 0.859, with relative orderings preserved.

B. Per-Feature Ablation

To quantify the contribution of each feature, we perform a leave-one-out ablation on the 200 medium-length QA pairs (Table VIII).

Two features drive the signal on academic text. **Discourse markers** (−27.8%) distinguish substantive paragraphs from filler; **numerical density** (−19.1%) captures statistics and quantitative results—precisely what factual questions target. Entity density and question presence show zero impact here, as expected: academic papers have uniform capitalization and few embedded questions. Feature importance is thus genre-dependent; narrative text likely ranks entity density higher.

TABLE VII
BUDGETMEM VS. NAIVE CHUNK SELECTION (30% BUDGET, 5K–10K TOKENS).

Method	F1 Score	Gap
Random 30%	0.6892	−14.3%
First 30%	0.7254	−9.8%
Last 30%	0.6734	−16.3%
TF-IDF Only	0.7689	−4.4%
BudgetMem (Ours)	0.8042	–

TABLE VIII
PER-FEATURE ABLATION (LEAVE-ONE-OUT) ON MEDIUM DOCUMENTS.

Configuration	F1 Score	F1 Drop
Full BudgetMem (all 6)	0.859	–
w/o Discourse markers	0.623	−27.8%
w/o Numerical density	0.697	−19.1%
w/o TF-IDF importance	0.858	−0.4%
w/o Position bias	0.860	−0.2%
w/o Entity density	0.859	0.0%
w/o Question presence	0.859	0.0%

C. Compression Granularity

The NarrativeQA experiments isolate compression granularity with ratio (30%), base model, and protocol held constant. Three regimes emerge: simple token-level ($F1 < 0.001$, catastrophic), neural token-level (0.0402, perplexity identifies load-bearing tokens), and simple chunk-level (0.0351, preserving paragraphs). Neural scoring rescues token-level compression $57\times$ ($0.0007 \rightarrow 0.0402$); switching token $\rightarrow$ chunk with simple scoring yields a comparable $50\times$ rescue ($0.0007 \rightarrow 0.0351$). Semantic coherence largely compensates for the absence of learned scoring.

D. Computational Cost

Table IX compares computational requirements. BudgetMem’s compression runs entirely on CPU using scikit-learn’s TF-IDF vectorizer, completing in ~ 3 seconds for a 50K-token document. LLMingua requires loading a BERT-based scoring model onto the GPU (~ 2 GB VRAM) and running inference over the entire document, taking ~ 45 seconds. The total pipeline time for BudgetMem (10.5s) is comparable to uncompressed RAG (9.8s) and $5\times$ faster than LLMingua (52.7s). Processing 1,000 documents takes ~ 2.9 hours with BudgetMem versus 14.6 hours with LLMingua.

For commercial APIs, storage reduction translates directly to token savings. At GPT-4o pricing (\$2.50/M input tokens), a 10K-token query costs $\sim \$0.025$; with

TABLE IX
COMPUTATIONAL COST (50K-TOKEN DOCUMENT, T4 GPU).

Component	Baseline	LLMLingua	BudgetMem
Compression	–	~ 45 s (GPU)	~ 3 s (CPU)
Indexing	1.2s	0.4s	0.4s
Retrieval	0.1s	0.1s	0.1s
Generation	8.5s	7.2s	7.0s
Total	9.8s	52.7s	10.5s
GPU for compression?	–	Yes	No
VRAM overhead	–	+2GB	0

BudgetMem retaining 30% of chunks and BM25 returning the top 3, effective context shrinks to ~ 450 tokens— $\sim \$0.001$ per query, a $25\times$ reduction.

E. Deployment Considerations

Prefer LLMingua when a GPU is available and maximizing F1 on ultra-long documents is paramount (e.g., medical or legal discovery). Prefer BudgetMem under resource constraints (edge devices, low-budget labs) or when compression decisions must be auditable (regulated industries). As a heuristic: full-context below 1K tokens, selective memory above 5K, with the crossover around 2K–3K tokens.

VII. CONCLUSION

We presented BudgetMem, a training-free selective memory system for cost-efficient long-context processing. By scoring document chunks with interpretable features and discarding low-salience material at ingestion time, BudgetMem matches baseline quality on structured documents while reducing index storage by 70%, and dramatically outperforms neural token-level compression on the same data—all without trained models, GPU-based compression, or expensive hardware.

Across four benchmarks including real academic papers, three findings emerge. First, compression granularity matters profoundly: token-level compression catastrophically degrades retrieval-based QA (LLMLingua-2 drops 38% F1 on medium documents; token-level TF-IDF scores near-zero on ultra-long texts), while chunk-level selection preserves or even improves performance. Second, simple interpretable features—particularly discourse markers and numerical density—provide sufficient signal for effective chunk selection on structured text. Third, BudgetMem generalizes to real academic papers (Qasper) without domain adaptation.

Limitations. Our medium-length evaluation uses controlled academic papers; results may not transfer to all real-world texts. Evaluation covers only extractive

QA on English; generalization to summarization, multi-hop reasoning, or multilingual settings remains untested. Our primary evaluation uses a 3B-parameter model; we replicate the central medium-document comparison on a 7B model from a different family (Table III), but a full multi-benchmark sweep at larger scales remains future work. We use BM25 rather than dense retrieval; dense retrievers may reduce the need for pre-retrieval filtering. Feature weights are manually tuned; learned policies would likely improve performance at the cost of the training-free property.

Future work. Learned write policies could close remaining gaps while remaining cheaper than full neural compression. Adaptive per-document budgets based on content complexity could improve over fixed ratios. Hybrid compression applying token-level refinement within retained chunks could capture the best of both granularity levels. Broader evaluation on summarization and multilingual benchmarks would clarify generalization.

REFERENCES

- [1] OpenAI, “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [2] Anthropic, “The claude 3 model family: Opus, sonnet, haiku,” *Anthropic Technical Report*, 2024.
- [3] A. Dubey, A. Jauhri, A. Pandey *et al.*, “The llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [4] R. Child, S. Gray, A. Radford, and I. Sutskever, “Generating long sequences with sparse transformers,” *arXiv preprint arXiv:1904.10509*, 2019.
- [5] M. Zaheer, G. Guruganesh, K. A. Dubey *et al.*, “Big bird: Transformers for longer sequences,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 17 283–17 297, 2020.
- [6] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret, “Transformers are rnns: Fast autoregressive transformers with linear attention,” *International Conference on Machine Learning*, pp. 5156–5165, 2020.
- [7] K. Choromanski, V. Likhoshesterov, D. Dohan *et al.*, “Rethinking attention with performers,” *International Conference on Learning Representations*, 2021.
- [8] Z. Dai, Z. Yang, Y. Yang *et al.*, “Transformer-xl: Attentive language models beyond a fixed-length context,” *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 2978–2988, 2019.
- [9] J. Su, Y. Lu, S. Pan *et al.*, “Roformer: Enhanced transformer with rotary position embedding,” *arXiv preprint arXiv:2104.09864*, 2021.
- [10] O. Press, N. A. Smith, and M. Lewis, “Train short, test long: Attention with linear biases enables input length extrapolation,” *International Conference on Learning Representations*, 2022.
- [11] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, “Llmllingua: Compressing prompts for accelerated inference of large language models,” *arXiv preprint arXiv:2310.05736*, 2023.
- [12] H. Jiang, Q. Wu, X. Luo, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu, “Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression,” *arXiv preprint arXiv:2310.06839*, 2023.
- [13] Y. Li, B. Dong, F. Guerin, and C. Lin, “Compressing context to enhance inference efficiency of large language models,” *arXiv preprint arXiv:2310.06201*, 2023.
- [14] F. Xu, W. Shi, and E. Choi, “Recomp: Improving retrieval-augmented lms with compression and selective augmentation,” *arXiv preprint arXiv:2310.04408*, 2023.
- [15] A. Chevalier, A. Wettig, A. Ajith, and D. Chen, “Adapting language models to compress contexts,” *arXiv preprint arXiv:2305.14788*, 2023.
- [16] A. Vaswani, N. Shazeer, N. Parmar *et al.*, “Attention is all you need,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [17] I. Beltagy, M. E. Peters, and A. Cohan, “Longformer: The long-document transformer,” *arXiv preprint arXiv:2004.05150*, 2020.
- [18] J. W. Rae, A. Potapenko, S. M. Jayakumar *et al.*, “Compressive transformers for long-range sequence modelling,” *arXiv preprint arXiv:1911.05507*, 2019.
- [19] P. Lewis, E. Perez, A. Piktus *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [20] V. Karpukhin, B. Oğuz, S. Min *et al.*, “Dense passage retrieval for open-domain question answering,” *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pp. 6769–6781, 2020.
- [21] G. Izacard and E. Grave, “Leveraging passage retrieval with generative models for open domain question answering,” *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, pp. 874–880, 2021.
- [22] W. Shi, S. Min, M. Yasunaga *et al.*, “Replug: Retrieval-augmented black-box language models,” *arXiv preprint arXiv:2301.12652*, 2023.
- [23] S. Borgeaud, A. Mensch, J. Hoffmann *et al.*, “Improving language models by retrieving from trillions of tokens,” *International Conference on Machine Learning*, pp. 2206–2240, 2022.
- [24] W. Wang, Z. Dong, J. Jiao *et al.*, “Memlong: Memory-augmented retrieval for long text generation,” *arXiv preprint arXiv:2408.16967*, 2024.
- [25] C. Xiao, P. Zhang, X. Hu *et al.*, “Inflm: Unveiling the intrinsic capacity of llms for understanding extremely long sequences with training-free memory,” *arXiv preprint arXiv:2402.04617*, 2024.
- [26] H. Wang, G. Zhou *et al.*, “Memorag: Moving towards next-gen rag via memory-inspired knowledge discovery,” *arXiv preprint arXiv:2409.05591*, 2024.
- [27] Y. Zhang, J. Chen *et al.*, “A-mem: Agentic memory for llm agents,” *arXiv preprint arXiv:2502.12110*, 2025.
- [28] P. Rajpurkar, R. Jia, and P. Liang, “Know what you don’t know: Unanswerable questions for SQuAD,” in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics*, 2018, pp. 784–789.
- [29] P. Dasigi, K. Lo, I. Beltagy *et al.*, “A dataset of information-seeking questions and answers anchored in research papers,” *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics*, pp. 4599–4610, 2021.
- [30] T. Kočiský, J. Schwarz, P. Blunsom *et al.*, “The narrativeqa reading comprehension challenge,” *Transactions of the Association for Computational Linguistics*, vol. 6, pp. 317–328, 2018.